RELATÓRIO TÉCNICO DE ARQUITETURA S.O.L.I.D.

MY Service Parking — Plataforma de Gestão de Estacionamento em Microsserviços

Autor: Yan Carlos Farias | Avaliação N2 (Apresentação 06/11) | Modelo C4 + Clean Architecture

Data de Emissão: 27/10/2026 | Escopo: 5 Microsserviços + API Gateway

1. RESUMO EXECUTIVO E OBJETIVOS DA REFATORAÇÃO

Este documento apresenta o mapeamento arquitetural e as refatorações aplicadas no código-fonte do projeto MY Service Parking para atender integralmente aos 5 princípios S.O.L.I.D. (SRP, OCP, LSP, ISP e DIP), mantendo consistência estrita com os diagramas do Modelo C4 (Containers, Componentes e Código/Classes). Foram eliminadas God Classes, cadeias rígidas de if/else, Fat Interfaces e acoplamentos diretos de infraestrutura nas camadas de negócio.

Sigla	Princípio S.O.L.I.D.	Padrão / Abstração Aplicada	Benefício Direto no Projeto
S (SRP)	**Single Responsibility Principle (Responsabilidade Única)**	Domain Calculators dedicados e separação Controller / Service / Entity / Repository	Classes coesas; cálculos financeiros e validações isolados da orquestração HTTP.
O (OCP)	**Open/Closed Principle (Aberto/Fechado)**	Strategy Pattern em Tarifas, Pagamentos, Acesso e Compatibilidade de Vagas	Novas tarifas, meios de pagamento ou regras de portaria são adicionadas sem editar código existente.
L (LSP)	**Liskov Substitution Principle (Substituição de Liskov)**	Contratos polimórficos uniformes em todas as Strategies e Repositories	Qualquer estratégia ou repositório pode ser substituído em runtime sem quebrar o chamador.
I (ISP)	**Interface Segregation Principle (Segregação de Interfaces)**	Interfaces granulares (IMensalistaClient, IPagamentoClient, IJwtService, IPasswordHasher)	Nenhum serviço depende de métodos que não utiliza (eliminação de Fat Interfaces).
D (DIP)	**Dependency Inversion Principle (Inversão de Dependência)**	Constructor Injection em 100% dos Services/Controllers + Composition Root nas rotas	Regras de negócio dependem apenas de abstrações (interfaces), facilitando testes e manutenção.

2. DIAGNÓSTICO TÉCNICO: O QUE FOI IDENTIFICADO E ALTERADO (ANTES VS. DEPOIS)

Princípio	Como Estava Antes (Problema Identificado)	O Que Foi Alterado / Refatorado (Solução S.O.L.I.D.)
SRP (Responsabilidade Única)	• EstacionamentoService.obterResumo() acumulava filtragem de datas, soma de receitas, consulta de pagamentos e cálculo de taxa de ocupação com divisor fixo (20). • MensalistaService não isolava a validação algorítmica de dígitos verificadores de CPF.	• Criado ResumoEstacionamentoCalculator.js (Domain Service puro) para isolar toda a consolidação financeira e de ocupação. • Criado ValidadorCpf.js no mensalista-service com o cálculo oficial dos dígitos verificadores. • Mantido DisponibilidadeCalculator.js no vaga-service.
OCP (Aberto para Extensão, Fechado para Modificação)	• EstacionamentoService.registrarEntrada() usava condicionais diretas para autorizar ou barrar mensalistas e rotativos. • Tarifação tinha apenas estratégias básicas e a alocação de vagas não possuía motor extensível de compatibilidade por porte.	• Criado RegrasAcesso.js (IRegraAcessoStrategy, RegraAcessoMensalista, RegraAcessoRotativo e AvaliadorAcessoContext). • Adicionada TarifaNoturnaStrategy em TarifaStrategy.js sem alterar nenhuma linha das tarifas existentes. • Criado CompatibilidadeVagaStrategy.js (AlocadorVagasEngine) no vaga-service.
LSP (Substituição de Liskov)	• Risco de métodos assíncronos/síncronos divergirem no retorno entre diferentes processadores de pagamento ou estratégias de tarifa.	• Padronizado o contrato de retorno de todas as subclasses de ITarifaStrategy ({ valorCalculado, dentroCarencia, tabelaAplicada }), IPagamentoProcessor ({ status, troco, detalhes }) e IRegraAcessoStrategy.
ISP (Segregação de Interfaces)	• Risco de clientes HTTP entre microsserviços exporem métodos desnecessários de CRUD.	• Segregado IMensalistaClient (apenas verificarMensalistaPorPlaca) e IPagamentoClient (apenas listarPagamentos e buscarPagamentoPorMovimentacao). • Segregado IJwtService de IPasswordHasher em SecurityServices.js.

Princípio	Como Estava Antes (Problema Identificado)	O Que Foi Alterado / Refatorado (Solução S.O.L.I.D.)
DIP (Inversão de Dependência)	• Necessidade de garantir que novas classes de domínio (calculadores e avaliadores) não fossem instanciadas de forma rígida dentro dos métodos dos Services.	• Atualizados os construtores de EstacionamentoService, MensalistaService e VagaService para receberem ResumoEstacionamentoCalculator, AvaliadorAcessoContext, ValidadorCpf e AlocadorVagasEngine via Constructor Injection nos arquivos routes/index.js (Composition Root).

3. RELAÇÃO COMPLETA DE ARQUIVOS CRIADOS E REFATORADOS

Arquivo / Caminho	Microsserviço	Ação	Princípios e Responsabilidade
src/domain/services/ResumoEstacionamentoCalculator.js	estacionamento-service	**NOVO**	SRP: Consolidação de métricas diárias, faturamento e ocupação.
src/domain/strategies/RegrasAcesso.js	estacionamento-service	**NOVO**	OCP / LSP: Strategy Pattern para autorização de entrada no pátio.
src/services/EstacionamentoService.js	estacionamento-service	**REFATORADO**	SRP / DIP: Delegação para calculador e avaliador injetados.
src/routes/index.js	estacionamento-service	**REFATORADO**	DIP: Composition Root instanciando e injetando dependências.
src/domain/services/ValidadorCpf.js	mensalista-service	**NOVO**	SRP: Validação algorítmica isolada de CPF.
src/services/MensalistaService.js	mensalista-service	**REFATORADO**	SRP / DIP: Uso do ValidadorCpf injetado via construtor.
src/routes/index.js	mensalista-service	**REFATORADO**	DIP: Injeção explícita no Composition Root.
src/domain/strategies/TarifaStrategy.js	pagamento-service	**REFATORADO**	OCP / LSP: Adição da TarifaNoturnaStrategy (pernoite R$ 35).
src/domain/strategies/PaymentProcessors.js	pagamento-service	**CONSOLIDADO**	OCP / SRP: Strategies para PIX (EMVCo), Cartão (Luhn) e Dinheiro.
src/domain/strategies/CompatibilidadeVagaStrategy.js	vaga-service	**NOVO**	OCP / SRP: Strategy de compatibilidade entre porte e tipo de vaga.
src/services/VagaService.js	vaga-service	**REFATORADO**	DIP / OCP: Injeção do AlocadorVagasEngine e filtro de vagas compatíveis.
src/routes/index.js	vaga-service	**REFATORADO**	DIP: Composition Root atualizado com AlocadorVagasEngine.
DIAGRAMA-C4/c4-componentes.puml & c4-codigo.puml	Documentação C4	**ATUALIZADO**	Alinhamento 1:1 entre diagramas C4 (Níveis 3 e 4) e código-fonte.

4. CHECKLIST DE CONFORMIDADE ACADÊMICA (ROTEIRO DA AULA DE 06/11)

#	Critério de Avaliação Solicitado (SOLID.md)	Status	Evidência no Código do MY Service Parking
01	**Cada componente do C4 Nível 3 possui módulo/pacote correspondente no código?**	**ATENDIDO**	Estrutura padronizada em controllers/, services/, domain/entities/, domain/services/, domain/strategies/, repositories/ e clients/.
02	**A camada de Domínio está isolada de Banco de Dados, APIs Externas e Frameworks?**	**ATENDIDO**	Entidades e Calculadores de domínio são classes JS puras, sem dependência de Express (req/res) ou chamadas de rede.
03	**As relações desenhadas no C4 coincidem com as injeções/interfaces do código?**	**ATENDIDO**	Diagramas c4-componentes.puml e c4-codigo.puml refletem exatamente os construtores e interfaces implementadas.
04	**As classes possuem responsabilidade única (sem formatação JSON ou SQL nos services)?**	**ATENDIDO**	Controllers cuidam de JSON/HTTP; Services orquestram casos de uso; Calculadores processam métricas; Repositories isolam dados.
05	**Evitou-se o surgimento de "God Classes"?**	**ATENDIDO**	Responsabilidades divididas em 5 microsserviços independentes e subcomponentes especialistas pequenos e coesos.
06	**Validação, cálculo e persistência estão delegados**	**ATENDIDO**	ValidadorCpf, RegrasAcesso, ResumoEstacionamentoCalculator,

#	Critério de Avaliação Solicitado (SOLID.md)	Status	Evidência no Código do MY Service Parking
	a componentes específicos?		DisponibilidadeCalculator, TarifaEngine e Repositories.
07	**Novas regras podem ser adicionadas criando novas classes sem alterar código existente?**	**ATENDIDO**	Demonstrado com TarifaNoturnaStrategy, PixProcessor/CartaoProcessor e RegraAcessoMensalista (Strategy Pattern).
08	**Subclasses não lançam exceções inesperadas (NotImplemented) para métodos da interface?**	**ATENDIDO**	Todas as classes concretas implementam integralmente os métodos de suas interfaces base respeitando o LSP.
09	**As interfaces são coesas e específicas em vez de uma única Fat Interface?**	**ATENDIDO**	IMensalistaClient, IPagamentoClient, IJwtService e IPasswordHasher possuem apenas os métodos estritamente necessários.
10	**Classes de alto nível dependem de abstrações/interfaces e são injetadas no ponto de entrada?**	**ATENDIDO**	Constructor Injection em todos os Services e instanciação centralizada no Composition Root (routes/index.js).